

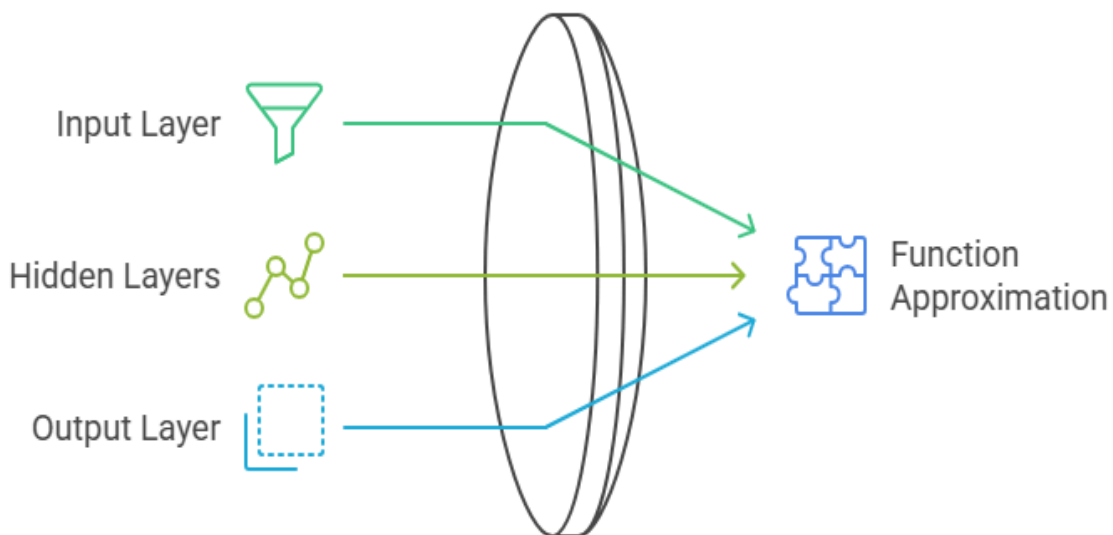
Multi-Layer Perceptron (MLP): Fundamentals of Neural Networks

Introduction

A Multi-Layer Perceptron (MLP) is one of the simplest and most fundamental types of artificial neural networks. It is a feedforward, fully connected network where information flows only forward — from input layer → hidden layers → output layer.

- **Key idea:** An MLP learns to approximate complex functions by applying a series of linear transformations followed by non-linear activation functions.
- MLPs are widely used as the foundation for more advanced architectures (CNNs, RNNs, Transformers).

MLP Architecture



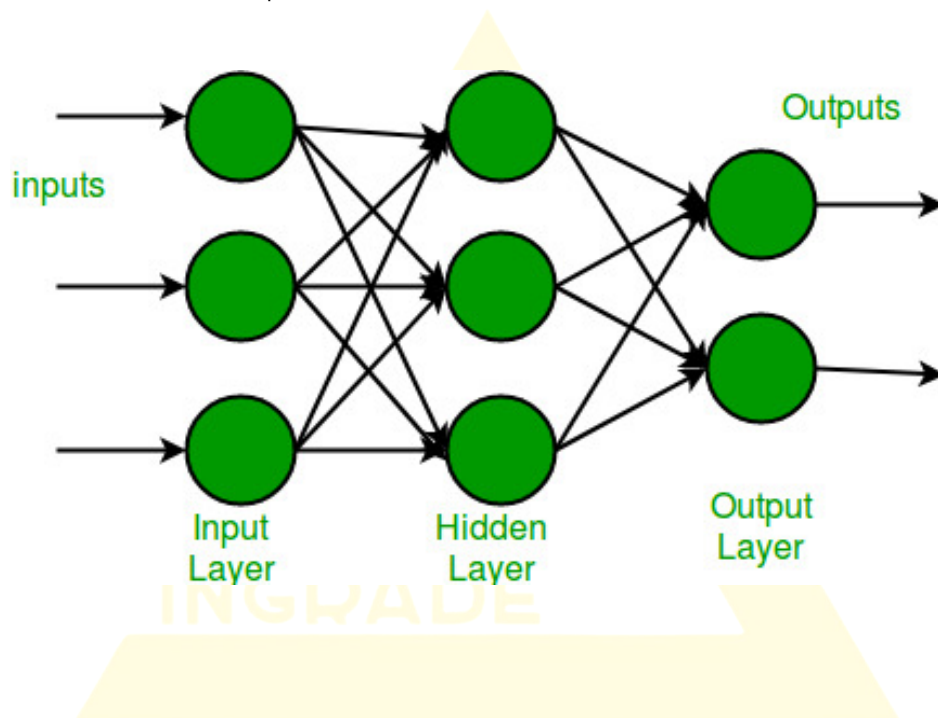
Analogy:

Think of MLP as a company:

- Input layer = interns (collect raw data).
- Hidden layers = managers (process & refine data).
- Output layer = CEO (makes the final decision).

Architecture of MLP

An MLP has three main components:

**2.1 Input Layer**

- Receives raw features from the dataset.
- Example: An image of size 28×28 pixels → 784 input nodes.

2.2 Hidden Layers

- Intermediate processing layers that transform inputs into abstract representations.
- Each neuron in a hidden layer:
 - Computes weighted sum of inputs.
 - Passes result through an activation function.
- Multiple hidden layers = ability to learn complex, non-linear functions.

2.3 Output Layer

- Produces the final prediction.
- Choice of activation depends on task:
 - Regression: Linear activation.
 - Binary Classification: Sigmoid.
 - Multi-class Classification: Softmax.

Mathematical Formulation

Each neuron follows:

$$h=f(Wx+b)$$

Where:

- **x** = input vector
- **W** = weights (learned importance of each feature)
- **b** = bias (adjusts threshold of activation)

- **f** = activation function (ReLU, sigmoid, tanh, etc.)
- **h** = neuron output

3.1 Layer-Wise Computation

- First hidden layer:

$$h_1 = f(W_1x + b_1)$$

- Second hidden layer:

$$h_2 = f(W_2h_1 + b_2)$$

- Final output:

$$y^{\wedge} = f_{out}(W_3h_2 + b_3)$$

Key insight: By stacking layers, an MLP can approximate any continuous function (Universal Approximation Theorem).

INGRADE

Importance of Activation Functions

Without activation → MLP reduces to a simple linear model.

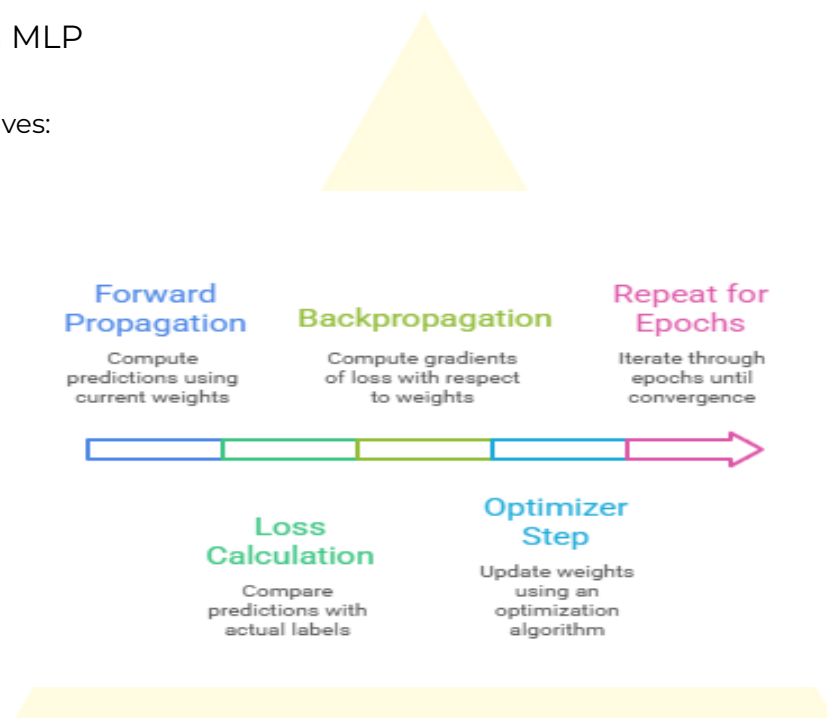
With activation → MLP captures non-linear patterns.

Common activation functions:

- Sigmoid → squashes values between 0 and 1 (good for probabilities).
- Tanh → outputs between -1 and 1 (centered around 0).
- ReLU (Rectified Linear Unit) → efficient, reduces vanishing gradient problem.

Training an MLP

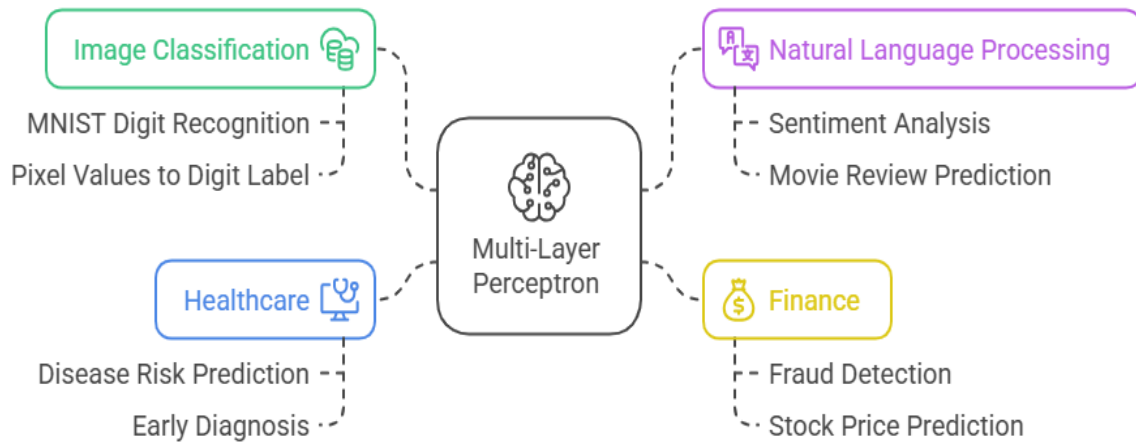
Training involves:



1. Forward Propagation → compute predictions.
2. Loss Calculation → compare predictions vs actual labels.
3. Backpropagation → compute gradients of loss wrt weights.
4. Optimizer Step → update weights (SGD, Adam, etc.).
5. Repeat for multiple epochs until convergence.

Applications of MLP

MLPs are widely used in early and modern AI systems:



6.1 Image Classification

- Example: MNIST digit recognition (0–9).
- Input = pixel values → Output = digit label.

6.2 Natural Language Processing (NLP)

- Sentiment analysis: Predict whether a movie review is positive or negative.

6.3 Finance

- Fraud detection in credit card transactions.
- Predict stock price trends using historical data.

6.4 Healthcare

- Predicting disease risk from patient records.

- Early diagnosis from structured medical data.

Strengths and Limitations

Strengths

- Simple and versatile.
- Can approximate any function.
- Foundation for advanced networks.

Limitations

- Struggles with high-dimensional structured data (like images).
- Requires careful tuning of hyperparameters.
- Training can be slow without optimizations.

Key Takeaways

- MLP = basic deep learning building block.
- Consists of input, hidden, and output layers.
- Learn via weighted sums + activation functions.
- Used in vision, NLP, finance, healthcare.
- Despite limitations, it paved the way for modern neural architectures.